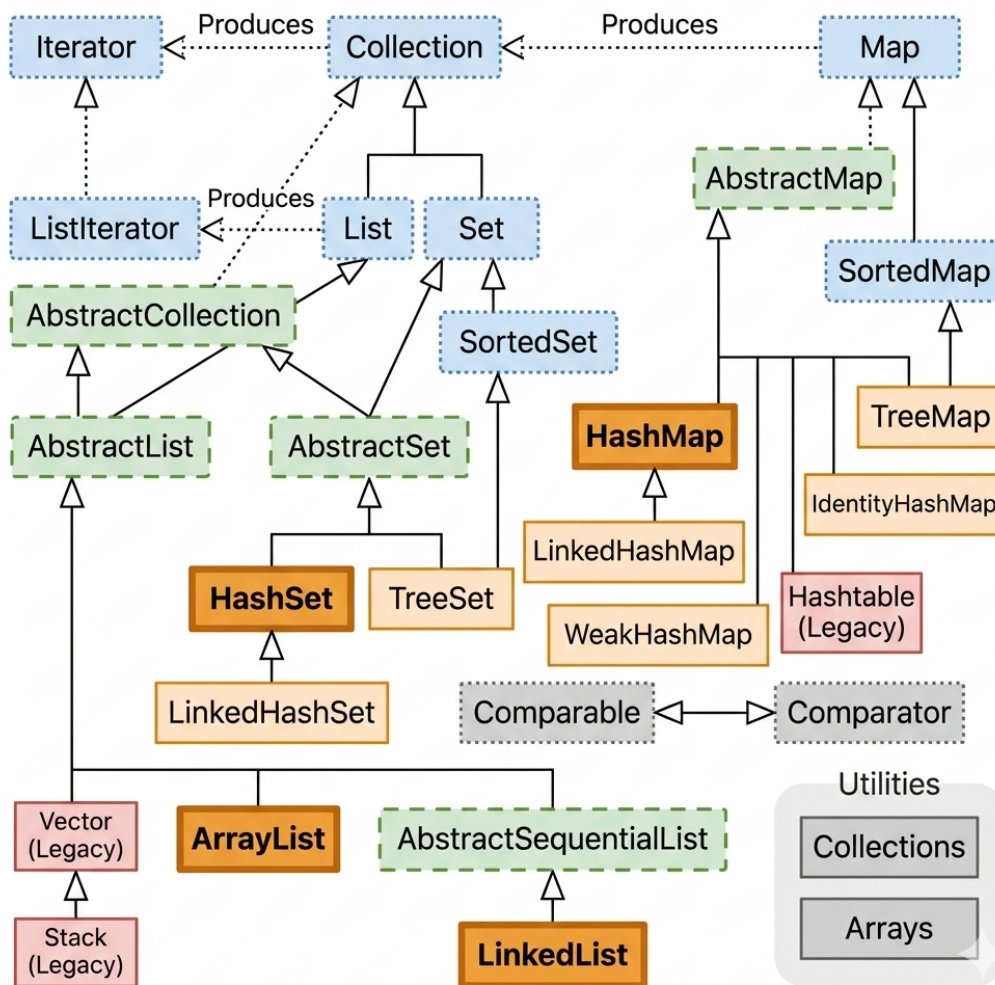


Java Collections Framework

Gerarchia, metodi distintivi e codice reale

Scheda di ripasso pronta da stampare



Idea chiave: dichiara le variabili usando l'interfaccia necessaria, e scegli la classe concreta in base a ordine, duplicati e prestazioni.

```
List<String> nomi = new ArrayList<>();
```

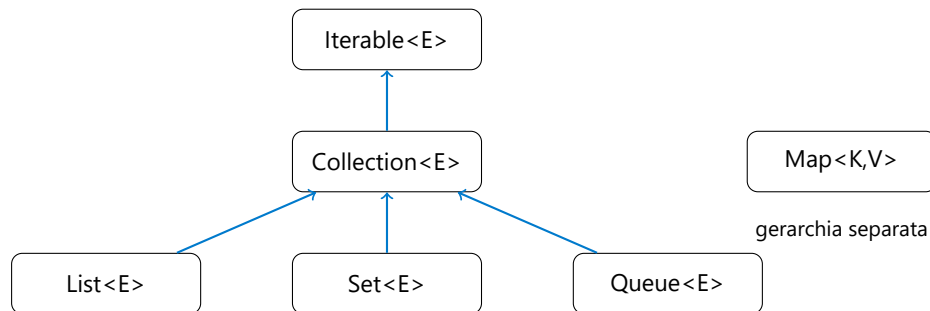
Indice

1	Come leggere la gerarchia	2
2	Iterable e Iterator	2
3	Collection: operazioni comuni	3
4	List, ArrayList e LinkedList	4
4.1	List	4
4.2	ArrayList	4
4.3	LinkedList, Queue e Deque	5
5	Set, HashSet e TreeSet	5
5.1	Set	5
5.2	HashSet	6
5.3	TreeSet	6
6	Map, HashMap e TreeMap	7
6.1	Map	7
6.2	HashMap	8
6.3	TreeMap	9
7	Comparable e Comparator	10
8	Utility statiche: Collections e Arrays	10
8.1	Collections	11
8.2	Arrays	12
9	Stream, lambda e Collectors	13
9.1	Come funziona una pipeline Stream	13
9.2	Operazioni terminali usate negli esami	14
9.3	flatMap e Collectors	15
10	Altre utility ricorrenti	15
10.1	Objects.requireNonNull	15
11	Tabella di scelta rapida	16
12	Errori tipici d'esame	16

1 Come leggere la gerarchia

Interfacce, classi astratte e classi concrete

- **Blu nell'immagine:** interfacce, cioè contratti che possono essere usati come tipi.
- **Verde:** classi astratte, cioè basi comuni per implementare nuove collezioni.
- **Arancione:** classi concrete, normalmente istanziate con `new`.
- **Map è separata da Collection:** gestisce associazioni chiave-valore, non una sequenza di elementi singoli.



2 Iterable e Iterator

Ruolo e comandi

`Iterable<E>` espone `iterator()` e può quindi essere usato in un `for-each`. L'oggetto `Iterator<E>` mantiene la posizione raggiunta durante la visita.

Comandi: `iterator()`, `hasNext()`, `next()`, `remove()` (se supportato dall'iteratore) e `forEach(action)`.

Visita manuale con Iterator

```
1 List<String> names = Arrays.asList("Anna", "Luca", "Sara");
2 Iterator<String> iterator = names.iterator();
3
4 // L'Iterator ricorda autonomamente la posizione corrente
5 while (iterator.hasNext()) {
6     String name = iterator.next();
7     System.out.println(name); // Anna, poi Luca, poi Sara
8 }
9
10 // Forma equivalente e più comune:
11 for (String name : names) {
12     System.out.println(name);
13 }
14
15 // Lambda: esegue println una volta per ogni elemento
16 names.forEach(name -> System.out.println(name));
17
18 // Riferimento a metodo: forma abbreviata equivalente
19 names.forEach(System.out::println);
```

Da ricordare

`Iterable` garantisce soltanto la possibilità di scorrere gli elementi: non possiede `size()` o `add()`. Questi metodi sono definiti da `Collection`.

3 Collection: operazioni comuni

Metodi usati nel corso

<code>add(e), addAll(c)</code>	aggiungono uno o più elementi; restituiscono se la collezione è cambiata
<code>remove(e), removeAll(c)</code>	rimuovono elementi; restituiscono se la collezione è cambiata
<code>retainAll(c)</code>	conserva soltanto gli elementi presenti anche in c
<code>contains(e), containsAll(c)</code>	verificano la presenza di uno o più elementi
<code>size(), isEmpty()</code>	restituiscono la dimensione e indicano se la collezione è vuota
<code>clear()</code>	elimina tutti gli elementi
<code>toArray(), iterator()</code>	convertono in array o permettono la visita

Programmare verso l'interfaccia Collection

```
1 Collection<Integer> values = new ArrayList<>();
2 values.addAll(Arrays.asList(1, 3, 5, 7));
3
4 boolean added = values.add(9);    // true: 9 viene inserito
5 boolean removed = values.remove(3); // true: 3 era presente
6 boolean found = values.contains(5); // true
7
8 System.out.println(values);        // [1, 5, 7, 9]
9 System.out.println(values.size()); // 4
10
11 for (int value : values) {
12     System.out.println(value);
13 }
```

4 List, ArrayList e LinkedList

4.1 List

Metodi d'istanza distintivi di List

Una List ammette duplicati, conserva l'ordine degli elementi e permette l'accesso tramite indice. Il primo indice è zero.

<code>get(index)</code>	legge l'elemento in posizione <code>index</code>
<code>set(index, value)</code>	sostituisce e restituisce il vecchio elemento
<code>add(index, value)</code>	inserisce spostando a destra gli elementi successivi
<code>remove(index)</code>	rimuove per posizione e restituisce l'elemento rimosso
<code>remove(value)</code>	rimuove la prima occorrenza del valore e restituisce un booleano
<code>indexOf(value)</code>	prima posizione, oppure <code>-1</code>
<code>lastIndexOf(value)</code>	ultima posizione, oppure <code>-1</code>
<code>subList(from, to)</code>	vista dall'indice <code>from</code> incluso a <code>to</code> escluso
<code>listIterator()</code>	iteratore bidirezionale specifico delle liste

Operazioni distintive di List

```

1 List<String> list = new ArrayList<>(
2     Arrays.asList("A", "B", "A")
3 ); // ordine e duplicati conservati
4
5 String second = list.get(1);    // "B"
6 String old = list.set(1, "X");  // old = "B"
7 list.add(1, "C");              // [A, C, X, A]
8 int firstA = list.indexOf("A"); // 0
9 int lastA = list.lastIndexOf("A");// 3
10 List<String> part = list.subList(1, 3); // [C, X]
11 // part è una vista: le modifiche si riflettono anche su list

```

Attenzione a remove con List<Integer>

`numbers.remove(1)` interpreta 1 come indice.
Per rimuovere il valore intero 1 bisogna scrivere `numbers.remove(Integer.valueOf(1))`.

4.2 ArrayList

Quando sceglierla

Array dinamico: `get(i)` e `set(i,e)` sono **$O(1)$** ; `add(e)` in fondo è mediamente **$O(1)$** , mentre inserimenti e rimozioni nel mezzo sono **$O(n)$** . È la scelta iniziale più comune quando serve una lista.

Metodi propri: `ensureCapacity(n)` prepara spazio senza cambiare `size()`; `trimToSize()` riduce la capacità interna agli elementi presenti. Tutti gli altri comandi principali provengono da List.

ArrayList: capacità e accesso rapido

```

1 ArrayList<String> students = new ArrayList<>();
2 students.ensureCapacity(30); // capacità >= 30, ma size() == 0
3
4 students.add("Anna");
5 students.add("Luca");
6 students.add("Sara");
7
8 String first = students.get(0); // accesso O(1): "Anna"
9 students.remove(1);            // [Anna, Sara]
10 students.trimToSize();         // riduce solo lo spazio interno

```

4.3 LinkedList, Queue e Deque

Quando sceglierla

Lista doppiamente concatenata: l'accesso per indice è **O(n)**, mentre inserimenti e rimozioni alle estremità sono **O(1)**. Implementa sia List sia Deque.

Metodi d'istanza di Queue e Deque

offer(e)	prova a inserire in coda; restituisce false se non può
poll()	legge e rimuove la testa; null se vuota
peek()	legge la testa senza rimuoverla; null se vuota
remove() / element()	come poll/peek, ma lanciano eccezione se vuota
addFirst(e), addLast(e)	inseriscono a una delle estremità
pollFirst(), pollLast()	rimuovono da un'estremità; null se vuota
peekFirst(), peekLast()	leggono un'estremità senza rimuovere
push(e), pop()	uso LIFO: inseriscono e rimuovono dalla testa

LinkedList usata come coda FIFO

```

1 Queue<String> queue = new LinkedList<>();
2 queue.offer("A");
3 queue.offer("B");
4 queue.offer("C"); // ingresso: A, B, C
5
6 String first = queue.peek(); // "A", la coda non cambia
7 String out1 = queue.poll(); // esce "A"
8 String out2 = queue.poll(); // esce "B"
9 System.out.println(queue); // [C]
```

Deque usata come pila LIFO

```

1 Deque<String> deque = new LinkedList<>();
2 deque.addLast("B"); // [B]
3 deque.addLast("C"); // [B, C]
4 deque.push("A"); // [A, B, C]: equivale ad addFirst
5
6 String top = deque.pop(); // "A", resta [B, C]
7 String last = deque.removeLast(); // "C", resta [B]
```

5 Set, HashSet e TreeSet

5.1 Set

Collezione senza duplicati

Set non aggiunge molti metodi a Collection, ma ne cambia il contratto: non ammette duplicati e non possiede indici. HashSet riconosce i duplicati tramite equals/hashCode; TreeSet usa invece compareTo o il Comparator.

Metodi usati: add, addAll, remove, contains, size, isEmpty, clear, iterator. add(e) restituisce false quando l'elemento era già presente.

5.2 HashSet

Caratteristiche

Non garantisce alcun ordine di iterazione. add, remove e contains costano mediamente **O(1)**. Usa hashCode() per restringere la ricerca ed equals() per verificare l'uguaglianza.

HashSet: unicità e ricerca rapida

```
1 Set<String> tags = new HashSet<>();
2 boolean first = tags.add("java"); // true
3 boolean second = tags.add("oop"); // true
4 boolean duplicate = tags.add("java"); // false
5
6 System.out.println(tags.size()); // 2
7 System.out.println(tags.contains("oop")); // true
8 tags.remove("oop"); // resta soltanto "java"
9
10 // L'ordine di stampa non è garantito
11 System.out.println(tags);
```

5.3 TreeSet

Caratteristiche

Mantiene gli elementi ordinati secondo l'ordine naturale (Comparable) oppure secondo un Comparator. Le operazioni principali costano **O(log n)**. Implementa NavigableSet.

Metodi d'istanza distintivi di TreeSet

first(), last()	minimo e massimo; eccezione se il set è vuoto
lower(e), higher(e)	elemento strettamente minore o maggiore
floor(e), ceiling(e)	minore/maggiore ammettendo anche l'uguaglianza
pollFirst(), pollLast()	rimuovono e restituiscono minimo o massimo; null se vuoto
headSet(to)	vista degli elementi minori di to
tailSet(from)	vista degli elementi maggiori o uguali a from
subSet(from, to)	vista da from incluso a to escluso
descendingSet()	vista con ordinamento inverso

TreeSet: ordine e ricerca per vicinanza

```
1 TreeSet<Integer> numbers = new TreeSet<>();
2 numbers.addAll(Arrays.asList(40, 10, 30, 20, 20));
3
4 System.out.println(numbers); // [10, 20, 30, 40]
5 // Ordinato automaticamente e senza duplicati
6
7 System.out.println(numbers.lower(20)); // 10: strettamente <
8 System.out.println(numbers.floor(20)); // 20: <=
9 System.out.println(numbers.ceiling(25)); // 30: >=
10 System.out.println(numbers.higher(30)); // 40: strettamente >
```

6 Map, HashMap e TreeMap

6.1 Map

Coppie chiave-valore

`Map<K,V>` non estende `Collection`: associa ogni chiave a un valore. Le chiavi sono uniche; una nuova chiamata a `put` con una chiave già presente sostituisce il valore precedente.

<code>put(key, value)</code>	inserisce e restituisce il vecchio valore
<code>putAll(map)</code>	copia tutte le associazioni
<code>get(key)</code>	valore associato; <code>null</code> se la chiave manca o è associata a <code>null</code>
<code>getOrDefault(key, def)</code>	valore associato, oppure <code>def</code> se la chiave manca
<code>putIfAbsent(key, value)</code>	inserisce se la chiave manca o è associata a <code>null</code>
<code>remove(key)</code>	elimina una chiave e restituisce il vecchio valore
<code>containsKey(key)</code>	verifica la presenza di una chiave
<code>containsValue(value)</code>	verifica la presenza di un valore
<code>compute(key, function)</code>	ricalcola il valore usando chiave e vecchio valore
<code>computeIfAbsent(key, function)</code>	calcola un valore se la chiave manca o vale <code>null</code>
<code>merge(key, value, function)</code>	inserisce il valore oppure lo combina con quello esistente
<code>keySet()</code>	vista <code>Set<K></code> delle chiavi
<code>values()</code>	vista <code>Collection<V></code> dei valori
<code>entrySet()</code>	vista delle coppie <code>Map.Entry<K,V></code>
<code>size(), isEmpty(), clear()</code>	numero di associazioni, controllo del vuoto e cancellazione

Se la funzione passata a `compute` restituisce `null`, l'associazione viene rimossa oppure non viene creata.

Map: inserimento, accesso e viste

```

1 Map<String, Integer> ages = new HashMap<>();
2 ages.put("Anna", 20);
3 ages.put("Luca", 25);
4 Integer oldAge = ages.put("Anna", 21); // restituisce 20
5
6 int anna = ages.get("Anna");           // 21
7 int sara = ages.getOrDefault("Sara", 0); // 0
8 boolean known = ages.containsKey("Luca"); // true
9 // containsKey distingue una chiave assente da una chiave associata a null
10
11 for (Map.Entry<String, Integer> entry : ages.entrySet()) {
12     // entry rappresenta una coppia chiave-valore
13     System.out.println(entry.getKey() + " -> " + entry.getValue());
14 }
```

6.2 HashMap

Caratteristiche

Non garantisce l'ordine delle chiavi. `get`, `put`, `remove` e `containsKey` costano mediamente **O(1)**. Applica `hashCode()` ed `equals()` alle chiavi. È la scelta iniziale più comune quando serve una mappa. Non aggiunge comandi fondamentali a `Map`: cambia soprattutto l'implementazione e le prestazioni.

HashMap e compute: conteggio delle frequenze

```
1 Map<String, Integer> frequency = new HashMap<>();
2 List<String> words = Arrays.asList("java", "oop", "java", "java");
3
4 for (String word : words) {
5     frequency.compute(word, (key, oldValue) ->
6         oldValue == null ? 1 : oldValue + 1
7     );
8     // Chiave assente: oldValue è null, quindi inserisce 1
9     // Chiave presente: incrementa il vecchio contatore
10 }
11
12 System.out.println(frequency.get("java")); // 3
13 System.out.println(frequency); // ordine non garantito
```

Alternative utili: putIfAbsent, computeIfAbsent e merge

```
1 Map<String, List<Integer>> grades = new HashMap<>();
2
3 grades.putIfAbsent("Anna", new ArrayList<>());
4 // Inserisce la lista se "Anna" manca oppure è associata a null
5
6 grades.computeIfAbsent("Luca", ignored -> new ArrayList<>()).add(28);
7 // Se "Luca" manca, crea la lista e poi aggiunge 28
8
9 Map<String, Integer> points = new HashMap<>();
10 points.merge("Anna", 3, (oldValue, value) -> oldValue + value);
11 points.merge("Anna", 1, Integer::sum);
12 // Prima chiamata: Anna=3. Seconda chiamata: Anna=4
```

6.3 TreeMap

Caratteristiche

Mantiene le chiavi ordinate. Le operazioni principali costano **$O(\log n)$** e richiedono chiavi Comparable oppure un Comparator. I metodi di navigazione appartengono a NavigableMap: per invocarli, il tipo della variabile deve essere NavigableMap o TreeMap, non soltanto Map.

Metodi d'istanza distintivi di TreeMap

firstKey(), lastKey()	prima e ultima chiave
lowerKey(k), higherKey(k)	chiave strettamente minore o maggiore
floorKey(k), ceilingKey(k)	chiave vicina ammettendo l'uguaglianza
firstEntry(), lastEntry()	prima e ultima coppia chiave-valore
pollFirstEntry(), pollLastEntry()	restituiscono e rimuovono una coppia estrema
headMap(k), tailMap(k)	viste delle chiavi minori di k o maggiori/uguali a k
subMap(from, to)	vista da from incluso a to escluso
descendingMap()	vista con chiavi in ordine inverso

TreeMap: chiavi automaticamente ordinate

```
1 Map<String, Integer> scores = new TreeMap<>();
2 scores.put("Zoe", 28);
3 scores.put("Anna", 30);
4 scores.put("Luca", 25);
5 scores.put("Anna", 31); // sostituisce il voto precedente
6
7 System.out.println(scores);
8 // {Anna=31, Luca=25, Zoe=28}: ordine naturale delle String
```

7 Comparable e Comparator

Due modi di definire l'ordine

Comparable<T>: definisce dentro la classe un unico ordine naturale tramite `compareTo`.

Comparator<T>: definisce all'esterno un criterio di ordinamento; per lo stesso tipo se ne possono creare diversi. Il confronto restituisce un numero negativo se il primo elemento viene prima, zero se i due elementi sono equivalenti per l'ordinamento e un numero positivo se il primo viene dopo.

`TreeSet` e `TreeMap` considerano duplicati due elementi o chiavi quando il confronto restituisce zero, anche se `equals()` restituisce `false`.

Comparable: ordine naturale dentro Person

```
1 class Person implements Comparable<Person> {
2     private final String name;
3     private final int birthYear;
4
5     Person(String name, int birthYear) {
6         this.name = name;
7         this.birthYear = birthYear;
8     }
9
10    public String getName() {
11        return this.name;
12    }
13
14    @Override
15    public int compareTo(Person other) {
16        return Integer.compare(this.birthYear, other.birthYear);
17    }
18 }
```

Comparator: ordine esterno alternativo

```
1 Comparator<Person> byName = new Comparator<>() {
2     @Override
3     public int compare(Person first, Person second) {
4         return first.getName().compareTo(second.getName());
5     }
6 };
7
8 Set<Person> ordered = new TreeSet<>(byName);
```

8 Utility statiche: Collections e Arrays

Attenzione: statici oppure d'istanza?

`list.add()`, `set.contains()` e `map.put()` sono metodi **d'istanza**: operano su uno specifico oggetto.

`Collections.sort(list)` e `Arrays.sort(array)` sono invece metodi **statici**: appartengono alle classi di utilità e ricevono la struttura su cui lavorare come argomento.

8.1 Collections

Metodi statici principali

<code>sort(list)</code>	ordina secondo l'ordine naturale
<code>sort(list, comparator)</code>	ordina usando un criterio esterno
<code>binarySearch(list, key)</code>	restituisce l'indice, oppure un valore negativo se l'elemento manca
<code>reverse(list)</code>	inverte l'ordine degli elementi
<code>shuffle(list)</code>	permuta casualmente gli elementi
<code>rotate(list, distance)</code>	ruota gli elementi di alcune posizioni
<code>swap(list, i, j)</code>	scambia due posizioni
<code>fill(list, value)</code>	sostituisce ogni elemento con lo stesso valore
<code>copy(dest, source)</code>	sovrascrive dest, che deve avere già almeno <code>source.size()</code> elementi
<code>replaceAll(list, old, new)</code>	sostituisce tutte le occorrenze
<code>min(collection),</code> <code>max(collection)</code>	trova minimo e massimo
<code>frequency(collection, value)</code>	conta quante volte compare un valore
<code>disjoint(c1, c2)</code>	verifica che non esistano elementi comuni
<code>nCopies(n, value)</code>	restituisce una lista non modificabile con n copie

Collections: ordinamento e ricerca

```

1 List<Integer> list = new ArrayList<>(Arrays.asList(4, 1, 3, 2));
2 Collections.sort(list); // [1, 2, 3, 4]
3
4 // La lista deve essere già ordinata con lo stesso criterio
5 int position = Collections.binarySearch(list, 3); // indice 2
6
7 Collections.reverse(list); // [4, 3, 2, 1]
8 Collections.swap(list, 0, 1); // [3, 4, 2, 1]
9 Collections.shuffle(list); // nuovo ordine casuale

```

Collections: interrogazioni e modifiche

```

1 List<Integer> values = new ArrayList<>(
2     Arrays.asList(3, 1, 3, 2)
3 );
4
5 int max = Collections.max(values); // 3
6 int times = Collections.frequency(values, 3); // 2
7 boolean separate = Collections.disjoint(
8     values, Arrays.asList(8, 9)
9 ); // true: nessun elemento comune
10
11 Collections.replaceAll(values, 3, 7); // [7, 1, 7, 2]
12 List<String> copies = Collections.nCopies(3, "Java");
13 // copies = [Java, Java, Java], lista non modificabile

```

8.2 Arrays

Metodi statici principali

<code>sort(array)</code>	ordina tutto l'array
<code>sort(array, from, to)</code>	ordina da from incluso a to escluso
<code>binarySearch(array, key)</code>	cerca in un array già ordinato
<code>fill(array, value)</code>	riempie tutte le celle con un valore
<code>fill(array, from, to, value)</code>	riempie da from incluso a to escluso
<code>copyOf(array, length)</code>	crea una copia con nuova lunghezza
<code>copyOfRange(array, from, to)</code>	copia da from incluso a to escluso
<code>equals(a, b)</code>	confronta due array elemento per elemento
<code>deepEquals(a, b)</code>	confronto ricorsivo di array annidati
<code>toString(array)</code>	rappresentazione leggibile di un array
<code>deepToString(array)</code>	rappresentazione di array multidimensionali
<code>asList(elements)</code>	restituisce una lista a dimensione fissa collegata all'array

Arrays: ordinamento, ricerca e riempimento

```

1 int[] values = {4, 1, 3, 2};
2 Arrays.sort(values); // modifica lo stesso array
3 System.out.println(Arrays.toString(values));
4 // [1, 2, 3, 4]
5
6 int position = Arrays.binarySearch(values, 3); // indice 2
7 Arrays.fill(values, 1, 3, 9);
8 // [1, 9, 9, 4]: intervallo [from, to]

```

Arrays: copie, confronti e conversioni

```

1 int[] original = {1, 2, 3, 4};
2 int[] copy = Arrays.copyOf(original, 6);
3 // [1, 2, 3, 4, 0, 0]: celle nuove al valore di default
4
5 int[] part = Arrays.copyOfRange(original, 1, 3);
6 // [2, 3]: indice iniziale incluso, finale escluso
7
8 boolean same = Arrays.equals(
9     original, new int[]{1, 2, 3, 4}
10 ); // true: confronta il contenuto
11
12 int[][] matrix = {{1, 2}, {3, 4}};
13 System.out.println(Arrays.deepToString(matrix)); // [[1, 2], [3, 4]]
14
15 List<String> fixed = Arrays.asList("A", "B", "C");
16 fixed.set(0, "X"); // consentito: [X, B, C]
17 // fixed.add("D"); // UnsupportedOperationException: size fissa
18
19 int[] primitive = {1, 2, 3};
20 List<int> oneElement = Arrays.asList(primitive);
21 // Con un array primitivo crea una lista di size 1 contenente tutto l'array

```

9 Stream, lambda e Collectors

9.1 Come funziona una pipeline Stream

Sorgente, operazioni intermedie e operazione terminale

Uno `Stream<T>` non è una collezione e non contiene direttamente gli elementi: rappresenta una pipeline che li elabora a partire da una sorgente.

Lambda: una scrittura come `name -> name.length()` rappresenta una piccola funzione senza nome. A sinistra di `->` ci sono i parametri; a destra c'è l'operazione da eseguire o il valore da restituire.

Import da ricordare:

`collection.stream()` può essere chiamato senza importare `Stream`.

`Collectors.toList()` richiede `import java.util.stream.Collectors;`.

Se si dichiara una variabile `Stream<T>`, serve anche `import java.util.stream.Stream;`.

`import java.util.*;` non importa il sottopackage `java.util.stream`.

<code>collection.stream()</code>	crea lo stream dalla collezione
<code>filter(predicate)</code>	conserva gli elementi che soddisfano una condizione
<code>map(function)</code>	trasforma ogni elemento in un altro valore
<code>flatMap(function)</code>	trasforma ogni elemento in uno stream e poi li appiattisce
<code>mapToInt(function)</code>	produce valori <code>int</code> , permettendo <code>sum()</code>
<code>sorted()</code>	ordina secondo l'ordine naturale; accetta anche un <code>comparator</code>
<code>distinct()</code>	elimina i duplicati secondo <code>equals()</code>

Intermedie oppure terminali

`filter`, `map` e `flatMap` sono operazioni **intermedie**: costruiscono la pipeline e restituiscono un altro `Stream`, ma non elaborano gli elementi finché non arriva un'operazione terminale. `collect`, `count`, `sum` e `forEach` sono

terminali: producono il risultato e consumano lo `Stream`. Uno `Stream` consumato non può essere riutilizzato.

Stream: filtrare, trasformare e raccogliere

```
1 List<String> names = Arrays.asList("Anna", "Luca", "Alberto", "Sara");
2
3 List<String> result = names.stream() // sorgente
4     .filter(name -> name.startsWith("A")) // solo nomi con A
5     .map(name -> name.toUpperCase())      // trasforma ogni nome
6     .collect(Collectors.toList());        // operazione terminale
7
8 System.out.println(result); // [ANNA, ALBERTO]
9 System.out.println(names);  // La lista originale non cambia
```

9.2 Operazioni terminali usate negli esami

Produrre un risultato

<code>forEach(action)</code>	esegue un'azione per ogni elemento e non restituisce un valore
<code>collect(collector)</code>	raccoglie il risultato, ad esempio in una lista, set, mappa o stringa
<code>count()</code>	restituisce il numero di elementi come long
<code>min(comparator),</code> <code>max(comparator)</code>	restituiscono un <code>Optional<T></code>
<code>sum()</code>	somma uno stream numerico, ad esempio <code>IntStream</code>
<code>anyMatch(predicate)</code>	vero se almeno un elemento soddisfa la condizione
<code>allMatch(predicate)</code>	vero se tutti soddisfano la condizione
<code>findFirst(), findAny()</code>	restituiscono un elemento dentro un <code>Optional<T></code>

Stream numerico: `mapToInt`, `sum` e `count`

```

1 List<Integer> grades = Arrays.asList(18, 24, 30, 27);
2
3 int total = grades.stream()
4     .mapToInt(value -> value) // Stream<Integer> diventa IntStream
5     .sum();                  // 99
6
7 long passed = grades.stream()
8     .filter(value -> value >= 18)
9     .count();                // 4: count restituisce Long
10
11 boolean allPassed = grades.stream()
12     .allMatch(value -> value >= 18); // true

```

Optional: risultato che potrebbe non esistere

```

1 List<Integer> empty = new ArrayList<>();
2
3 int minimum = empty.stream()
4     .min(Integer::compare)
5     .orElse(-1);
6 // Lo stream è vuoto: min restituisce Optional.empty()
7 // orElse(-1) fornisce un valore alternativo sicuro
8
9 int found = Arrays.asList(3, 7, 10).stream()
10     .filter(value -> value % 2 == 0)
11     .findAny()
12     .orElse(-1); // 10, oppure -1 se nessun numero pari esiste

```

9.3 flatMap e Collectors

Metodi di Collectors più usati

Collectors.toList()	raccoglie gli elementi in una lista; il tipo concreto non è garantito
Collectors.toSet()	raccoglie in un set; tipo concreto e ordine non sono garantiti
Collectors.toMap(k, v)	crea una mappa; chiavi duplicate causano un'eccezione
Collectors.joining(sep)	concatena stringhe usando un separatore

flatMap: appiattare collezioni annidate

```

1 List<List<String>> groups = Arrays.asList(
2     Arrays.asList("Anna", "Luca"),
3     Arrays.asList("Sara", "Anna")
4 );
5
6 Set<String> uniqueNames = groups.stream()
7     .flatMap(group -> group.stream())
8     // Da Stream<List<String>> a un solo Stream<String>
9     .collect(Collectors.toSet());
10
11 System.out.println(uniqueNames); // ordine non garantito, senza duplicati

```

Collectors: creare Map e String

```

1 List<String> names = Arrays.asList("Anna", "Luca", "Sara");
2
3 Map<String, Integer> lengths = names.stream()
4     .collect(Collectors.toMap(
5         name -> name,           // chiave: il nome
6         name -> name.length()   // valore: la lunghezza
7     ));
8 // {Anna=4, Luca=4, Sara=4}: ordine non garantito
9
10 String text = names.stream()
11     .collect(Collectors.joining(", "));
12 // "Anna, Luca, Sara"

```

10 Altre utility ricorrenti

10.1 Objects.requireNonNull

Controllare subito gli argomenti

`Objects.requireNonNull(value)` è un metodo statico di `Objects`. Se il valore è `null`, lancia immediatamente `NullPointerException`; altrimenti restituisce il valore ricevuto. È utile nei costruttori per rifiutare subito argomenti non validi.

Validazione nel costruttore

```

1 class Student {
2     private final String name;
3
4     Student(String name) {
5         this.name = Objects.requireNonNull(
6             name, "Il nome non può essere null"
7         );
8         // Errore immediato nel punto in cui nasce l'oggetto
9     }
10 }

```

11 Tabella di scelta rapida

Tipo	Duplicati	Ordinamento	Quando usarlo
ArrayList	sì	inserimento	lista standard e accesso frequente tramite indice
LinkedList	sì	inserimento	coda/deque e operazioni frequenti alle estremità
HashSet	no	nessuno	elementi unici e ricerca rapida
TreeSet	no	naturale/Comparator	elementi unici, ordinati e navigabili
HashMap	chiavi uniche	nessuno	associazioni chiave-valore rapide
TreeMap	chiavi uniche	per chiave	associazioni con chiavi ordinate e navigabili

Regola pratica

Come scelta iniziale usa ArrayList, HashSet e HashMap. Scegli le varianti Tree* quando serve mantenere l'ordine, e Deque quando il lavoro avviene soprattutto alle estremità.

12 Errori tipici d'esame

- Chiamare `size()` su `Iterable`: il metodo appartiene a `Collection`.
- Cercare `get(i)` su un `Set`: i set non hanno indici.
- Scrivere `list.remove(1)` pensando di rimuovere il valore 1: su una lista di interi viene rimosso l'elemento di indice 1.
- Credere che `HashSet` o `HashMap` mantengano l'ordine di inserimento.
- Modificare campi usati da `equals/hashCode` mentre l'oggetto è un elemento di `HashSet` o una chiave di `HashMap`.
- Usare un tipo non confrontabile dentro `TreeSet` senza fornire un `Comparator`.
- Dimenticare che `Arrays.asList()` produce una lista a dimensione fissa e che un array primitivo diventa un solo elemento.
- Confondere `Map` con `Collection`: le viste sono `keySet()`, `values()` ed `entrySet()`.
- Riutilizzare uno `Stream` dopo un'operazione terminale: lo stream è già stato consumato.
- Confondere `map` con `flatMap`: il primo trasforma, il secondo trasforma e appiattisce.
- Chiamare `Optional.get()` senza verificarne il contenuto: preferire `orElse`, `orElseGet` o `ifPresent`.
- Credere che `import java.util.*` importi anche `Collectors`: serve `java.util.stream.Collectors`.